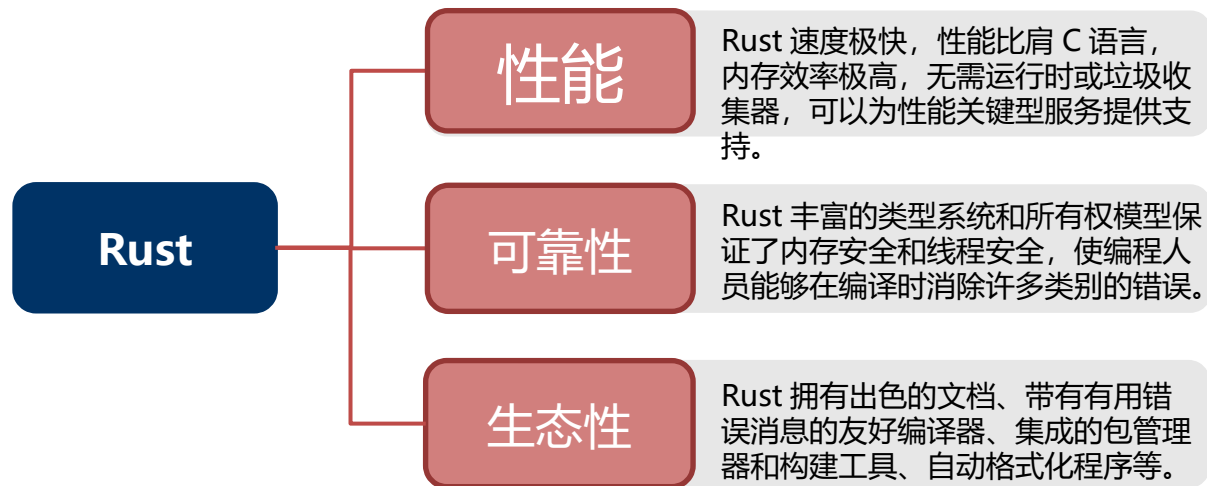




编译原理

编译原理课程设计

类 Rust 编译器实现



Rust 是一门能够安全高效地编写系统级程序的语言，在内存安全方面具有显著优势。



知名问答平台 StackOverflow 的调查表明，自 2015 年以来，Rust 一直是开发者最爱的编程语言。

nature

《Nature》杂志 2020 年尾的文章《Why Scientists are Turning to Rust》中也强调：科学家极为推崇 Rust。

华为是五位 Rust 基金会创始白金会员成员之一，引领了 Rust 语言的发展。

华为内部使用

- Rust 广泛用于嵌入式系统开发、系统驱动、云计算、虚拟存储、网络传输协议、并发编程框架基础库等产品中。
- 华为实验室正基于 Rust 探索先进的代码解析、安全分析等工具。

Rust 社区贡献

- 华为深度参与了在 Rust 社区中，贡献了包括 C 到 Rust 转换、内联汇编、交叉编译、Parking Lot 并发库、SIMD 基础库、文档导航、代码多态化、热补丁、AOP 等特性。

Rust 中国推广

- 华为战略支持了在中国举办的 Rust China Conf 大会，并推行多项社区活动。
- 华为也为中国的开发者提供 Rust 教程和 Rust 编码规范等。

课程设计的目的

- 1. 掌握使用高级程序语言实现一个一遍完成的、简单语言的编译器的方法;
- 2. 掌握简单的词法分析器、语法分析器、符号表管理、中间代码生成以及目标代码生成的实现方法;
- 3. 掌握将生成代码写入文件的技术。

课程设计的要求

- 1. 使用高级程序语言作为实现语言，实现一个类 Rust 语言的编译器。编码实现编译器的组成部分。
- 2. 要求的类 Rust 编译器是个一遍的编译程序，词法分析程序作为子程序，需要的时候被语法分析程序调用；
- 3. 使用语法制导的翻译技术，在语法分析的同时生成中间代码，并保存到文件中。
- 4. 要求输入类 Rust 语言源程序，输出中间代码表示的程序；
- 5. 要求输入类 Rust 语言源程序，输出目标代码（可汇编执行）的程序。

提交的文档

- 1. 编译器源程序
- 2. 编译器可执行程序
- 3. 设计说明书
 - ① 系统方案设计说明
 - ② 程序功能描述
 - ③ 程序具体实现：主要算法、基本框图、主要模块、功能函数等
 - ④ 执行界面和运行结果
 - ⑤ 设计中遇到的问题及解决方法或设计体会
 - ⑥ AI 辅助编程相关历程、启示、心得等（如有）

- 需求分析能力 20%
- 系统方案（算法）设计能力 20%
- 编程能力 30%
- 撰写报告能力 30%
- 完成每组可得到对应★，凑齐至少 7 颗★可以申请优秀，否则至多为良。

□ 2.3、4.2、4.3 :	★ ★
□ 5.2、5.3、5.4 :	★ ★
□ 6.1、6.2、6.3、6.4 :	★ ★ ★ ★
□ 7.1、7.2、7.3、7.4、7.5 :	★ ★ ★
□ 8.1、8.2、8.3 :	★ ★ ★
□ 9.1、9.2、9.3 :	★ ★ ★

考核方法

- **设计报告 + 答辩**

- **提交一个压缩包，包括：设计说明书（电子版）、源程序、可执行程序**
- **申请优秀者须参加线上答辩**
 - **线上答辩时间：讲解 5 分钟 + 演示（录屏） 1 分钟 + 提问 2-4 分钟**

上交形式和时间

- 含源代码、可执行代码和设计说明书文档的压缩包
- 提交时间：截止 7 月 31 日
- 线上答辩时间： 8 月 3 日

类 Rust 语法和语义规则

➤ 1.1 基础程序

- Program -> < 声明串 >
- < 声明串 > -> 空 | < 声明 > < 声明串 >
- < 声明 > -> < 函数声明 >
- < 函数声明 > -> < 函数头声明 > < 语句块 >
- < 函数头声明 > -> fn <ID> '(' < 形参列表 > ')'
- < 形参列表 > -> 空
- < 语句块 > -> '{' < 语句串 > '}'
- < 语句串 > -> 空

语义说明：定义了一个最基础的程序结构，支持声明若干个函数，每个函数无参数且函数体为空。

```
fn program_1_1() {  
  
}
```

➤ 0.1 变量属性

- < 变量属性 > -> mut

语义说明：允许变量被二次赋值，后续拓展的不可变变量不允许被二次赋值。

➤ 0.2 类型

- < 类型 > -> i32

语义说明：支持基础类型 i32，表示 32 位的整型。

➤ 0.3 左值

- < 左值 > -> < 可取引用 >
- < 可取引用 > -> < 可取元素 >
- < 可取元素 > -> <ID>

语义说明：标识符的一些属性：可作为左值、可取引用、可取元素。

类 Rust 语法和语义规则

➤ 1.2 语句 (拓展 1.1)

- `< 语句串 > -> < 语句 > < 语句串 >`
- `< 语句 > -> ';' ;`

语义说明：语句串由任意数量个语句构成，单个分号可以作为一个语句。

➤ 1.3 返回语句 (拓展 1.2)

- `< 语句 > -> < 返回语句 >`
- `< 返回语句 > -> return ';' ;`

语义说明：`< 返回语句 >` 用于从函数调用中退出，并将控制权交还给调用者。

目前不支持返回值。

➤ 1.4 函数输入 (依赖 0.1、0.2，拓展 1.1)

- `< 形参列表 > -> < 形参 > | < 形参 > ',' < 形参列表 >`
- `< 形参 > -> < 变量属性 > < ID > ':' < 类型 >`

语义说明：函数声明时接收参数，参数具有指定的类型。

➤ 1.5 函数输出 (依赖 0.2、3.1，拓展 1.1、1.3)

- `< 函数头声明 > -> fn < ID > '(' < 形参列表 > ')' '->' < 类型 >`
- `< 返回语句 > -> return < 表达式 > ';' ;`

语义说明：`< 函数头声明 >` 支持标注返回类型，`< 返回语句 >` 支持返回一个表达式的值。

```
fn program_1_2() {  
    .....  
}
```

```
fn program_1_3() {  
    return ;  
}
```

```
fn program_1_4(mut a:i32) {  
    .....  
}
```

```
fn program_1_5__1() -> i32 {  
    return 1;  
}
```

```
fn program_1_5__2() -> i32 {  
    return ;  
}
```

× 返回语句的类型 (空) 和函数声明返回类型 (i32) 不一致

```
fn program_1_5__3() {  
    return 1;  
}
```

× 返回语句的类型 (i32) 和函数声明返回类型 (空) 不一致

类 Rust 语法和语义规则

➤ 2.0 变量声明 (依赖 0.1、0.2)

- $\langle \text{变量声明} \rangle \rightarrow \langle \text{变量属性} \rangle \langle \text{ID} \rangle$
- $\langle \text{变量声明} \rangle \rightarrow \langle \text{变量属性} \rangle \langle \text{ID} \rangle ':' \langle \text{类型} \rangle$

语义说明：允许两种变量声明：

- 声明一个变量但不指定类型，后续需要对该变量进行赋值来支持编译器推导类型；
- 声明一个变量且指定类型。

➤ 2.1 变量声明语句 (依赖 2.0, 拓展 1.2)

- $\langle \text{语句} \rangle \rightarrow \langle \text{变量声明语句} \rangle$
- $\langle \text{变量声明语句} \rangle \rightarrow \text{let } \langle \text{变量声明} \rangle ';' \text{ ;}$

语义说明：变量声明语句需在变量声明前加上 `let` 关键字，变量允许被二次声明，每次声明隐藏之前的绑定，称为重影。

```
fn program_2_1_1() {  
    let mut a:i32;  
}
```

```
fn program_2_1_2() {  
    let mut b;  
    b=1;  
}
```

✓ 后续可以判断 b 的类型为整型

```
fn program_2_1_3() {  
    let mut b;  
}
```

✗ 后续无法推断 b 的类型

```
fn program_2_1_4() {  
    let mut a:i32;  
    a=1;  
    let mut a:i32;  
    a=2;  
}
```

✓ 可以二次声明, 称为重影

类 Rust 语法和语义规则

➤ 2.0 变量声明 (依赖 0.1、0.2)

- $\langle \text{变量声明} \rangle \rightarrow \langle \text{变量属性} \rangle \langle \text{ID} \rangle$
- $\langle \text{变量声明} \rangle \rightarrow \langle \text{变量属性} \rangle \langle \text{ID} \rangle ':' \langle \text{类型} \rangle$

语义说明：允许两种变量声明：

- 声明一个变量但不指定类型，后续需要对该变量进行赋值来支持编译器推导类型；
- 声明一个变量且指定类型。

➤ 2.2 赋值语句 (依赖 0.3、3.1, 拓展 1.2)

- $\langle \text{语句} \rangle \rightarrow \langle \text{赋值语句} \rangle$
- $\langle \text{赋值语句} \rangle \rightarrow \langle \text{左值} \rangle '=' \langle \text{表达式} \rangle ';'$

语义说明：允许将一个 $\langle \text{表达式} \rangle$ 的计算结果赋值给左值，从而实现变量值的修改。左值需提前声明；若左值已知类型需和表达式的类型一致；右值若为变量则需提前声明且赋值。

```
fn program_2_2__1(mut a:i32) {  
    a=32;  
}
```

```
fn program_2_2__2() {  
    a=32;  
}
```

× 变量未声明

```
fn program_2_2__3(mut a:i32) {  
    a=1==1;  
}
```

× 变量类型不一致

```
fn program_2_2__4() {  
    let mut b:i32=a;  
}
```

× 右值未提前声明

```
fn program_2_2__5() {  
    let mut a:i32;  
    let mut b:i32=a;  
}
```

× 右值未提前赋值

类 Rust 语法和语义规则

➤ 2.0 变量声明 (依赖 0.1、0.2)

- $\langle \text{变量声明} \rangle \rightarrow \langle \text{变量属性} \rangle \langle \text{ID} \rangle$
- $\langle \text{变量声明} \rangle \rightarrow \langle \text{变量属性} \rangle \langle \text{ID} \rangle ':' \langle \text{类型} \rangle$

语义说明：允许两种变量声明：

- 声明一个变量但不指定类型，后续需要对该变量进行赋值来支持编译器推导类型；
- 声明一个变量且指定类型。

➤ 2.3 变量声明赋值语句 (依赖 2.0、3.1，拓展 1.2)

- $\langle \text{语句} \rangle \rightarrow \langle \text{变量声明赋值语句} \rangle$
- $\langle \text{变量声明赋值语句} \rangle \rightarrow \text{let } \langle \text{变量声明} \rangle '=' \langle \text{表达式} \rangle ';' \text{ ;}$

语义说明：变量声明的同时进行赋值

- 如果变量声明时未指定类型，则类型根据后续表达式推断；
- 如果变量声明时指定类型，则需检测是否与后续表达式类型一致，不一致应报错。

变量允许被二次声明，每次声明隐藏之前的绑定，称为重影。

```
fn program_2_3_1() {  
    let mut a:i32=1;  
    let mut b=1;  
}
```

```
fn program_2_3_2() {  
    let mut a:i32=1;  
    let mut a=2;  
    let mut a:i32=3;  
}
```

✓ 可以二次声明，
称为重影

类 Rust 语法和语义规则

➤ 3.1 基本表达式 (依赖 0.3、拓展 1.2)

- $\langle \text{语句} \rangle \rightarrow \langle \text{表达式} \rangle ;$
- $\langle \text{表达式} \rangle \rightarrow \langle \text{加减表达式} \rangle$
- $\langle \text{加减表达式} \rangle \rightarrow \langle \text{项} \rangle$
- $\langle \text{项} \rangle \rightarrow \langle \text{因子} \rangle$
- $\langle \text{因子} \rangle \rightarrow \langle \text{NUM} \rangle \mid \langle \text{左值} \rangle \mid '(\langle \text{表达式} \rangle)'$

语义说明：提供表达式求值的能力，支持常量、变量以及带括号的嵌套表达式。

```
fn program_3_1_1() {  
    0;  
    (1);  
    ((2));  
    (((3)));  
}
```

```
fn program_3_1_2(mut a:i32) {  
    a;  
    (a);  
    ((a));  
    (((a)));  
}
```

类 Rust 语法和语义规则

➤ 3.2 增加比较运算（依赖 3.1、拓展 3.1）

- $\langle \text{表达式} \rangle \rightarrow \langle \text{表达式} \rangle \langle \text{比较运算符} \rangle \langle \text{加减表达式} \rangle$
- $\langle \text{比较运算符} \rangle \rightarrow '<' \mid '<=' \mid '>' \mid '>=' \mid '==' \mid '!='$

语义说明：支持比较运算，且优先级乘除 > 加减 > 比较。

```
fn program_3_2() {  
    1<2;  
    3<=4;  
    5>6;  
    7>=8;  
    9==10;  
    11!=12;  
}
```

➤ 3.3 增加加减运算（依赖 3.1、拓展 3.1）

- $\langle \text{加减表达式} \rangle \rightarrow \langle \text{加减表达式} \rangle \langle \text{加减运算符} \rangle \langle \text{项} \rangle$
- $\langle \text{加减运算符} \rangle \rightarrow '+' \mid '-'$

语义说明：支持加减运算，且优先级乘除 > 加减 > 比较。

```
fn program_3_3() {  
    1+2;  
    3-4;  
}
```

➤ 3.4 增加乘除运算（依赖 3.1、拓展 3.1）

- $\langle \text{项} \rangle \rightarrow \langle \text{项} \rangle \langle \text{乘除运算符} \rangle \langle \text{因子} \rangle$
- $\langle \text{乘除运算符} \rangle \rightarrow '*' \mid '/'$

语义说明：支持乘除运算，且优先级乘除 > 加减 > 比较。

```
fn program_3_4() {  
    1*2;  
    3/4;  
}
```

类 Rust 语法和语义规则

➤ 3.5 函数调用 (依赖 3.1 , 拓展 3.1)

- $\langle \text{因子} \rangle \rightarrow \langle \text{ID} \rangle '(\langle \text{实参列表} \rangle)'$
- $\langle \text{实参列表} \rangle \rightarrow \text{空} \mid \langle \text{表达式} \rangle \mid \langle \text{表达式} \rangle ',' \langle \text{实参列表} \rangle$

语义说明：提供函数调用表达式的能力，函数调用结果可以参与运算。实参数量与形参数量需一致； $\langle \text{实参列表} \rangle$ 中的每个 $\langle \text{表达式} \rangle$ 的类型必须与对应位置形参的类型一致；若函数无返回值，则不能参与运算或作为右值。

```
fn program_3_3_3_a() {  
    }  
fn program_3_3_3_b() {  
    program_3_3_3_a(1);  
}
```

× 实参数量与形参数量不一致

```
fn program_3_3_4_a(mut a:i32) {  
    }  
fn program_3_3_4_b() {  
    program_3_3_4_a(program_3_3_4_a);  
}
```

× 实参类型与形参类型不一致

```
fn program_3_3_1_a() {  
    }  
fn program_3_3_1_b() {  
    program_3_3_1_a();  
}  
  
fn program_3_3_2_a(mut a:i32) {  
    }  
fn program_3_3_2_b() {  
    program_3_3_2_a(1+2);  
}
```

```
fn program_3_3_5_a() {  
    }  
fn program_3_3_5_b() {  
    let mut a=program_3_3_5_a();  
}
```

× 无返回值函数不能作为右值

类 Rust 语法和语义规则

➤ 4.1 选择结构 (依赖 1.1、3.1, 拓展 1.2)

- `<语句> -> <if 语句>`
- `<if 语句> -> if <表达式> <语句块> <else 部分>`
- `<else 部分> -> 空`

语义说明: 支持 if 条件分支语句, 若 `<表达式>` 为真, 则执行 if 后的 `<语句块>`。

➤ 4.2 增加 else (依赖 1.1, 拓展 4.1)

- `<else 部分> -> else <语句块>`

语义说明: 拓展 else 分支, 若前一个 if 条件不成立, 则执行 else 后的 `<语句块>`。

➤ 4.3 增加 else if (依赖 1.1, 拓展 4.1)

- `<else 部分> -> else if <表达式> <语句块> <else 部分>`

语义说明: 拓展 else if 分支, 若前一个 if 或 else if 条件不成立, 则评估此分支的 `<表达式>`, 若为真则执行该 else if 后的 `<语句块>`。

```
fn program_4_1(mut a:i32) -> i32 {  
    if a>0 {  
        return 1;  
    }  
    return 0;  
}
```

```
fn program_4_2(mut a:i32) -> i32 {  
    if a>0 {  
        return 1;  
    } else {  
        return 0;  
    }  
}
```

```
fn program_4_3(mut a:i32) -> i32 {  
    if a>0 {  
        return a+1;  
    } else if a<0 {  
        return a-1;  
    } else {  
        return 0;  
    }  
}
```

类 Rust 语法和语义规则

➤ 5.0 循环语句 (依赖 5.1 or 5.2 or 5.3 , 拓展 1.2)

- `< 语句 > -> < 循环语句 >`

➤ 5.1 while 循环结构 (依赖 1.1 、 3.1 , 拓展 5.0)

- `< 循环语句 > -> <while 语句 >`
- `<while 语句 > -> while < 表达式 > < 语句块 >`

语义说明: 拓展循环控制能力, 实现基于条件的多次重复执行逻辑。每次循环开始时求表达式的值, 若结果为真, 则执行 `< 语句块 >`; 若为假, 则跳出循环。

➤ 5.2 for 循环结构 (依赖 1.1 、 2.0 、 3.1 , 拓展 5.0)

- `< 循环语句 > -> <for 语句 >`
- `<for 语句 > -> for < 变量声明内部 > in < 可迭代结构 > < 语句块 >`
- `< 可迭代结构 > -> < 表达式 > '..' < 表达式 >`

语义说明: 拓展基于范围的迭代能力, 对可迭代结构中的每一个元素进行遍历, `< 可迭代结构 >` 是左闭右开区间, 即包含第一个 `< 表达式 >` 的值, 不包含第二个 `< 表达式 >` 的值。 `< 表达式 >` 类型需要是整型。

```
fn program_5_1(mut n:i32) {  
    while n>0 {  
        n=n-1;  
    }  
}
```

```
fn program_5_2_1(mut n:i32) {  
    for mut i in 1..n+1 {  
        n=n-1;  
    }  
}  
fn program_5_2_2(mut n:i32) {  
    for mut i in 1..n+0.1 {  
        n=n-1;  
    }  
}
```

× `n+0.1` 类型不是整型

类 Rust 语法和语义规则

➤ 5.3 loop 循环结构 (依赖 1.1 , 拓展 5.0)

- < 循环语句 > -> < loop 语句 >
- < loop 语句 > -> loop < 语句块 >

语义说明: 拓展无条件无限循环的能力, 定义一个无限循环, 每次迭代时执行 < 语句块 > 中的内容, 只有通过显式退出指令 (如 break) 才能终止循环。

➤ 5.4 增加 break 和 continue (拓展 1.2)

- < 语句 > -> break ';' | continue ';' ;

语义说明: 增强循环逻辑的灵活性和表达能力。

- break ';' 表示立即终止最内层当前所在的循环, 控制流将跳出循环; 必须出现在循环体内; 在嵌套循环中使用仅跳出当前最内层循环。
- continue ';' 表示跳过当前循环体中剩余的代码, 控制流将跳转到下一次循环; 必须出现在循环体内; 在嵌套循环中使用仅影响当前所在循环。

```
fn program_5_3() {  
    loop {  
  
    }  
}
```

```
fn program_5_4_1() {  
    while 1==1 {  
        break;  
    }  
}
```

```
fn program_5_4_2() {  
    break;  
}
```

× break; 必须出现在循环体内

```
fn program_5_4_3() {  
    while 1==0 {  
        continue;  
    }  
}
```

```
fn program_5_4_4() {  
    continue;  
}
```

× continue; 必须出现在循环体内

类 Rust 语法和语义规则

➤ 6.1 变量不可变属性 (拓展 0.1)

- `< 变量属性 > -> 空`

语义说明: 变量声明时可以不标注 `mut` 关键字, 如此变量不可被二次赋值;
若变量是复合类型 (如数组), 其元素也不可被二次赋值。

➤ 6.2 不可变引用 (依赖 0.2、0.3, 拓展 0.2、0.3)

- `< 类型 > -> '&' < 类型 >`
- `< 可取引用 > -> '&' < 可取引用 >`

语义说明: `'&' < 类型 >` 表示一个指向对应数据类型的不可变引用类型。
`'&' < 可取引用 >` 表示创建一个变量的不可变引用; 允许通过该引用读取但不允许修改指向数据; 可以同时存在多个不可变引用。

➤ 6.3 可变引用 (依赖 0.2、0.3, 拓展 0.2、0.3)

- `< 类型 > -> '&' mut < 类型 >`
- `< 可取引用 > -> '&' mut < 可取引用 >`

语义说明: `'&' mut < 类型 >` 表示一个指向对应数据类型的可变引用类型。
`'&' mut < 可取引用 >` 表示创建一个变量的可变引用; 允许通过该引用修改指向数据; 不允许存在其他的不可变或可变引用; 仅允许从可变变量创建可变引

```
fn program_6_1_1() {  
    let a:i32=1;  
    let b=2;  
}
```

```
fn program_6_1_2() {  
    let c:i32=1;  
    c=2;  
}
```

× 不可变变量不可被二次赋值

```
fn program_6_2_1() {  
    let a:i32=1;  
    let b:&i32=&a;  
}
```

```
fn program_6_2_2() {  
    let mut a:i32=1;  
    let b=&a;  
    let c=&a;  
}
```

✓ 可以存在多个不可变引用

类 Rust 语法和语义规则

➤ 6.1 变量不可变属性 (拓展 0.1)

- `< 变量属性 > -> 空`

语义说明：变量声明时可以不标注 `mut` 关键字，如此变量不可被二次赋值；若变量是复合类型（如数组），其元素也不可被二次赋值。

➤ 6.2 不可变引用 (依赖 0.2、0.3，拓展 0.2、0.3)

- `< 类型 > -> '&' < 类型 >`
- `< 可取引用 > -> '&' < 可取引用 >`

语义说明：'`&`' `< 类型 >` 表示一个指向对应数据类型的不可变引用类型。
'`&`' `< 可取引用 >` 表示创建一个变量的不可变引用；允许通过该引用读取但不允许修改指向数据；可以同时存在多个不可变引用。

➤ 6.3 可变引用 (依赖 0.2、0.3，拓展 0.2、0.3)

- `< 类型 > -> '&' mut < 类型 >`
- `< 可取引用 > -> '&' mut < 可取引用 >`

语义说明：'`&`' `mut < 类型 >` 表示一个指向对应数据类型的可变引用类型。
'`&`' `mut < 可取引用 >` 表示创建一个变量的可变引用；允许通过该引用修改指向数据；不允许存在其他的不可变或可变引用；仅允许从可变变量创建可变引

```
fn program_6_3() {  
    let mut a:i32=1;  
    let mut b:&mut i32=&mut a;  
}
```

```
fn program_6_3_4() {  
    let mut a:i32=1;  
    let b=&a;  
    let mut c=&mut a;  
}
```

× 可变引用不能和其他的引用共存

```
fn program_6_3_5() {  
    let a:i32=1;  
    let mut b=&mut a;  
}
```

× 仅支持从可变变量创建可变引用

类 Rust 语法和语义规则

➤ 6.2 不可变引用 (依赖 0.2、0.3, 拓展 0.2、0.3)

- `< 类型 > -> '&' < 类型 >`
- `< 可取引用 > -> '&' < 可取引用 >`

语义说明: `'&' < 类型 >` 表示一个指向对应数据类型的不可变引用类型。

`'&' < 可取引用 >` 表示创建一个变量的不可变引用; 允许通过该引用读取但不允许修改指向数据; 可以同时存在多个不可变引用。

➤ 6.3 可变引用 (依赖 0.2、0.3, 拓展 0.2、0.3)

- `< 类型 > -> '&' mut < 类型 >`
- `< 可取引用 > -> '&' mut < 可取引用 >`

语义说明: `'&' mut < 类型 >` 表示一个指向对应数据类型的可变引用类

型。 `'&' mut < 可取引用 >` 表示创建一个变量的可变引用; 允许通过该引用修改指向数据; 不允许存在其他的不可变或可变引用; 仅允许从可变变量创建可变引用。

➤ 6.4 借用 (依赖 0.3、拓展 0.3)

- `< 左值 > -> '*' < 左值 >`

语义说明: 表示对一个引用类型的值进行解引用操作, 访问其指向的数据;

```
fn program_6_4_1() {  
    let mut a:i32=1;  
    let mut b=&mut a;  
    *b=2;  
}
```

```
fn program_6_4_2() {  
    let mut a:i32=1;  
    let mut b=*a; × 不允许对非引用  
                  类型进行解引用  
}
```

```
fn program_6_4_3() {  
    let mut a:i32=1;  
    let mut b=&a;  
    *b=2; × 不可变引用不可  
          以修改指向数据  
}
```

类 Rust 语法和语义规则

➤ 7.0 函数表达式块 (依赖 1.1、3.1, 拓展 3.1)

- `< 函数表达式语句块 > -> '{' < 函数表达式语句串 > '}'`
- `< 函数表达式语句串 > -> < 表达式 > | < 语句 > < 函数表达式语句串 >`

语义说明：允许将复杂的逻辑封装在一个块里，其中包含一系列 `< 语句 >` 和一个末尾的 `< 表达式 >` 的代码块，末尾的 `< 表达式 >` 的值作为返回值。

➤ 7.1 函数表达式块作为表达式 (依赖 7.0, 拓展 0.3)

- `< 可取元素 > -> < 函数表达式语句块 >`

语义说明：函数表达式语句块可以作为表达式参与计算，通常是作为右值赋值给变量。

➤ 7.2 函数表达式块作为函数体 (依赖 1.1、7.0, 拓展 1.1)

- `< 函数声明 > -> < 函数头声明 > < 函数表达式语句块 >`

语义说明：提供使用结构化表达式块作为函数执行内容的能力，一个完整的函数可由 `< 函数头 >` 和 `< 函数表达式语句块 >` 组成，`< 函数表达式代码块 >` 的最后一个 `< 表达式 >` 的值作为返回值。

```
fn program_7_1(mut x:i32,mu y:i32) {  
    let mut z={  
        let mut t=x*x+x;  
        t=t+x*y;  
        t  
    };  
}
```

```
fn program_7_2(mut x:i32,mu y:i32) -> i32 {  
    let mut t=x*x+x;  
    t=t+x*y;  
    t  
}
```

类 Rust 语法和语义规则

➤ 7.3 选择表达式 (依赖 3.1、7.0, 拓展 0.3)

- `< 表达式 > -> < 选择表达式 >`
- `< 选择表达式 > -> if < 表达式 > < 函数表达式语句块 > else < 函数表达式语句块 >`

语义说明: 提供将 if-else 结构用作表达式并返回值的能力, 一个 `< 表达式 >` 可以是 if-else 结构和 `< 函数表达式语句块 >` 的组合, 根据 if 后的 `< 表达式 >` 的真假, 决定执行哪一个 `< 函数表达式语句块 >`, 该语句块的返回值作为该 `< 选择表达式 >` 的返回值。

➤ 7.4 循环表达式 (依赖 3.1、5.3, 拓展 0.3)

- `< 表达式 > -> < loop 语句 >`
- `< 语句 > -> break < 表达式 > ;`

语义说明: 提供将循环结构作表达式并返回值的能力, 一个 `< 表达式 >` 可以是一个 `< loop 语句 >`, 其最终返回值由某个分支中的 `break < 表达式 >` 决定。break 必须出现在循环体内, 否则应报错; break 后的表达式必须能求值; 若多个 break 存在于同一循环体中, 它们必须返回相同类型。

```
fn program_7_3(mut a:i32) {  
    let mut b=if a>0 {  
        1  
    } else {  
        0  
    };  
}
```

```
fn program_7_4_1() {  
    let mut a=loop {  
        break 1;  
    };  
}
```

```
fn program_7_4_2() {  
    break 2;  
}
```

× break < 表达式 >;
必须出现在循环体内

类 Rust 语法和语义规则

➤ 8.1 数组类型 (依赖 0.2 , 拓展 0.2)

- $\langle \text{类型} \rangle \rightarrow '[' \langle \text{类型} \rangle ';' \langle \text{NUM} \rangle ']'$

语义说明：拓展数组类型，产生式右边的 $\langle \text{类型} \rangle$ 是数组中元素的类型， $\langle \text{NUM} \rangle$ 是数组的长度，必须是一个正整数常量。

➤ 8.2 数组表达式 (依赖 3.1 、 0.3 , 拓展 5.2)

- $\langle \text{可取元素} \rangle \rightarrow '[' \langle \text{数组元素列表} \rangle ']'$
- $\langle \text{数组元素列表} \rangle \rightarrow \text{空} \mid \langle \text{表达式} \rangle \mid \langle \text{表达式} \rangle ';' \langle \text{数组元素列表} \rangle$
- $\langle \text{可迭代结构} \rangle \rightarrow \langle \text{表达式} \rangle$

语义说明：数组是由方括号 '[' 包裹，逗号 ';' 分隔元素 $\langle \text{表达式} \rangle$ 的结构， $\langle \text{表达式} \rangle$ 的数量必须与数组的长度一致，每个 $\langle \text{表达式} \rangle$ 的类型要与数组元素类型一致。

```
fn program_8_1() {  
    let mut a:[i32;3];  
    let mut b:[i32;3];3;  
}
```

```
fn program_8_2_1(mut a:i32) {  
    let mut a:[i32;2];  
    a=[1,2,3];  
}
```

× 初始化时的元素数量与数组长度不一致

```
fn program_8_2_2() {  
    let mut a:[i32;1];1;  
    a=[1];  
}
```

× 初始化时元素的类型与数组的元素类型不一致

类 Rust 语法和语义规则

➤ 8.3 数组元素 (依赖 0.3、3.1, 拓展 0.3)

- `<可取元素> -> <可取元素> '[' <表达式> ']'`

语义说明：提供按索引访问数组元素的能力，元素可以被取元素、可以参与计算、可以作为左值被修改。索引 `<表达式>` 的类型必须是整型；索引 `<表达式>` 必须在合法范围 `[0, len)` 内；若数组变量不可变，则数组或其子元素不允许被二次赋值。

```
fn program_8_3_1(mut a:[[i32;3];3]) {  
    let mut b:i32=a[0][0];  
}
```

✓ 元素可以被取子元素

```
fn program_8_3_2(mut a:[i32;3]) {  
    let mut b:i32=a[0]+a[1]+a[2];  
}
```

✓ 元素可以参与计算

```
fn program_8_3_3(mut a:[i32;3]) {  
    a[0]=1;  
}
```

✓ 元素可以作为左值被修改

```
fn program_8_3_4(mut a:i32) {  
    let mut a=[1,2,3];  
    let mut b=a[a];  
}
```

✗ 数组的索引的类型
必须是整数类型

```
fn program_8_3_5() {  
    let mut a=[1,2,3];  
    let mut b=a[3];  
}
```

✗ 数组的索引必须在合
法范围 [0,3) 内

```
fn program_8_3_6() {  
    let a:[i32;3]=[1,2,3];  
    a[0]=4;  
}
```

✗ 不可变数组的元素也
不可变，不允许被二次
赋值

类 Rust 语法和语义规则

➤ 9.1 元组类型 (依赖 0.2 , 拓展 0.2)

- $\langle \text{类型} \rangle \rightarrow '(' \langle \text{元组类型内部} \rangle ')'$
- $\langle \text{元组类型内部} \rangle \rightarrow \text{空} \mid \langle \text{类型} \rangle ',' \langle \text{类型列表} \rangle$
- $\langle \text{类型列表} \rangle \rightarrow \text{空} \mid \langle \text{类型} \rangle \mid \langle \text{类型} \rangle ',' \langle \text{类型列表} \rangle$

语义说明：拓展元组类型，是由圆括号 '(' 包裹，逗号 ',' 分隔元素类型的结构。支持空元组类型 () 和非零数量元素的元组类型。

➤ 9.2 元组类型 (依赖 3.1 , 拓展 0.3)

- $\langle \text{可取元素} \rangle \rightarrow '(' \langle \text{元组赋值内部} \rangle ')'$
- $\langle \text{元组赋值内部} \rangle \rightarrow \text{空} \mid \langle \text{表达式} \rangle ',' \langle \text{元组元素列表} \rangle$
- $\langle \text{元组元素列表} \rangle \rightarrow \text{空} \mid \langle \text{表达式} \rangle \mid \langle \text{表达式} \rangle ',' \langle \text{元组元素列表} \rangle$

语义说明：元组是由圆括号 '(' 包裹，逗号 ',' 分隔元素 $\langle \text{表达式} \rangle$ 的结构，支持空元组 () 和带元素的元组表达式， $\langle \text{表达式} \rangle$ 的数量必须与元组类型中的 $\langle \text{类型} \rangle$ 数量一致；每个 $\langle \text{表达式} \rangle$ 的类型要与元组对应位置元素 $\langle \text{类型} \rangle$ 一致；仅有一个元素的元组的元素后必须要加逗号 ',' ，从而与加括号的表达式区分。

```
fn program_9_1() {  
    let a:();  
    let b:(i32,i32);  
    let a:();i32;  
}
```

```
fn program_9_2_1(mut a:i32) {  
    let mut a:(i32,i32);  
    a=(1,2,3);  
}
```

× 初始化时的元素数量与元组长度不一致

```
fn program_9_2_2() {  
    let mut a:();  
    a=(1,);  
}
```

× 初始化时元素的类型与元组定义的元素类型不一致

类 Rust 语法和语义规则

➤ 9.3 元组元素 (依赖 0.3 , 拓展 0.3)

- `< 可赋值元素 >->< 元素 > '!' <NUM>`

语义说明：提供按索引访问元组元素的能力，元素可以被取元素、可以参与计算、可以作为左值被修改。索引必须是整型字面量；索引必须在合法范围 `[0, len)` 内；若元组变量不可变，则元组或其子元素不允许被二次赋值。

```
fn program_9_3_1(mut a:((i32,i32),)) {  
    let mut b:i32=a.0.0;  
}
```

✓ 元素可以被取子元素

```
fn program_9_3_2(mut a:(i32,i32,i32)) {  
    let mut b:i32=a.0+a.1+a.2;  
}
```

✓ 元素可以参与计算

```
fn program_9_3_3(mut a:[i32;3]) {  
    a.0=1;  
}
```

✓ 元素可以作为左值被修改

```
fn program_9_3_4(mut a:i32) {  
    let mut a=(1,2,3);  
    let mut b=a.a;  
}
```

✗ 元组的索引的类型
必须是整数类型

```
fn program_9_3_5() {  
    let mut a=(1,2,3);  
    let mut b=a.3;  
}
```

✗ 元组的索引必须在
合法范围 `[0,3)` 内

```
fn program_9_3_6() {  
    let a:(i32,i32,i32)=(1,2,3)  
    a.0=4;  
}
```

✗ 不可变元组的元素也
不可变，不允许被二次
赋值

评分标准

实验评价内容	所占比重	要求
需求分析能力	20%	A（90 分以上）：对系统的可行性、用户需求和功能需求进行了准确完整的分析与总结； B（80-89）：对系统的可行性、用户需求和功能需求进行了较为完整的分析与总结； C（70-79）：对系统的可行性与基本功能需求进行了分析与总结； D（60-69）：对系统的基本功能需求进行了简单的分析与总结； E（60 分以下）：无需求分析内容

评分标准

实验评价内容	所占比重	要求
系统方案（算法）设计能力	20%	A（90 分以上）：系统设计与实现非常完善，概要设计和详细设计完整； B（80-89）：系统设计与实现较完善，概要设计和详细设计较完整； C（70-79）：设计并实现了系统的基本功能，概要设计和详细设计基本完整； D（60-69）：仅设计并实现较简单的系统； E（60 分以下）：无系统设计与实现。

评分标准

实验评价内容	所占比重	要求
编程能力	30%	A（90 分以上）：编程实现要求的全部功能，实现输出目标代码（可汇编执行）的程序，正确无误、界面友好；能在报告中透彻解释并论证 AI 生成代码的底层逻辑，具备极强的审查能力以修复复杂的逻辑缺陷。 B（80-89 分）：编程实现要求的全部功能，实现输出中间代码表示的程序，正确无误、界面友好；能在报告中清晰解释 AI 生成代码的实现原理，能独立审查并有效修复其中的逻辑缺陷。 C（70-79 分）：编程实现要求的全部功能，实现输出中间代码表示的程序，程序较正确、界面较友好；基本理解并在报告中说明 AI 生成代码的作用，仅能修复明显的逻辑缺陷。 D（60-69 分）：编程基本实现要求功能，实现输出中间代码表示的程序，程序基本正确、提供交互界面；无法在报告中清楚解释 AI 生成的代码，缺乏审查与修复逻辑缺陷的能力。 E（60 分以下）：未能编程实现要求的基本功能；完全无法理解或解释 AI 生成的代码，未能修复逻辑缺陷，无法保证程序的可运

评分标准

实验评价内容	所占比重	要求
撰写报告能力	30%	A（90 分以上）：表达通顺，逻辑清晰，内容完整，实验充分，提出个人想法，不存在抄袭；在报告中明确声明 AI 工具的使用环节与用途，能深度提炼并展现基于 AI 生成内容的独立思考与批判性见解。 B（80-89 分）：表达通顺，逻辑清晰，内容较为完整，实验较充分，不存在抄袭；如实说明 AI 工具的参与情况，能对 AI 生成的文本进行有效的审查、润色与重组，保证行文逻辑连贯。 C（70-79 分）：表达通顺，内容基本完整，不存在抄袭；能在报告中交代 AI 的使用情况，对 AI 生成内容的依赖度较高但经过了基本的核对与修改，无明显生硬拼接的痕迹。 D（60-69 分）：报告内容不完整，不存在抄袭；未能规范披露 AI 工具的使用细节，或直接套用 AI 生成的文本而缺乏必要的个人理解与实质性修改，导致部分内容脱离实际实验过程。 E（60 分以下）：存在大篇幅抄袭或严重学术不端行为；未做任何合规声明，直接将 AI 生成的整段报告作为个人成果提交，完全